

Week 1 Theory Notes: C for Embedded and STM32F401 Peripherals

EE0827 Embedded System Applications

Part 1: C for Embedded Systems

Why C for Embedded?

C is the dominant language for embedded systems because it provides:

- **Direct hardware access** — memory-mapped registers are just pointer dereferences
- **Predictable memory layout** — no hidden allocations, structs map to hardware
- **No hidden overhead** — unlike C++ virtual functions or garbage collection
- **Portability** — same code runs on different MCU families with minimal changes

Pointers

A pointer stores the **address** of a variable, not its value.

```
uint32_t value = 100;
uint32_t *ptr = &value; // ptr holds address of value
*ptr = 200;             // value is now 200
```

Key operations:

- `&variable` — get address of variable
- `*pointer` — dereference (access value at address)
- `pointer++` — increment by `sizeof(*pointer)` bytes

Pointer Arithmetic

When incrementing a pointer, it moves by the size of the pointed-to type:

Pointer Type	ptr++ moves by
uint8_t *	1 byte
uint16_t *	2 bytes
uint32_t *	4 bytes

Pointers to Registers

All peripheral access in embedded C works through pointers to specific memory addresses:

```
#define GPIOA_ODR ((volatile uint32_t*)0x40020014)

*GPIOA_ODR |= (1 << 5); // Set bit 5 (LED ON)
*GPIOA_ODR &= ~(1 << 5); // Clear bit 5 (LED OFF)
```

This is how all peripheral access works under the hood.

- Every HAL call eventually reads/writes a memory-mapped register
- Unlike a PC (where the OS randomizes addresses every run), MCU registers live at **fixed physical addresses** defined by the chip manufacturer
- 0x40020014 is *always* GPIOA's output register — on every board, every reset, forever

Bit Manipulation

Register configuration requires setting, clearing, toggling, and checking individual bits:

Operation	Syntax	Effect
Set a bit	<code>reg = (1 << n)</code>	Bit n = 1, others unchanged
Clear a bit	<code>reg &= ~(1 << n)</code>	Bit n = 0, others unchanged
Toggle a bit	<code>reg ^= (1 << n)</code>	Bit n flips
Check a bit	<code>if (reg & (1 << n))</code>	True if bit n is set

Read-Modify-Write: `|=` and `&=` read the register, modify one bit, then write back — leaving all other bits unchanged.

Structs

Structs group related data into a single type — essential for HAL handles and CMSIS register blocks:

```
typedef struct {
    uint32_t timestamp;
    uint16_t adc_value;
    uint8_t  sensor_id;
    uint8_t  status;
} sensor_reading_t;

sensor_reading_t reading;
sensor_reading_t *ptr = &reading;
ptr->timestamp = HAL_GetTick(); // arrow operator for pointer access
```

Every HAL peripheral handle (`huart2`, `htim2`) is a struct accessed via pointer.

The `volatile` Keyword

`volatile` tells the compiler that a variable can change outside the current code path — so it must always read from memory, never from a cached register.

Use `volatile` for:

- Hardware registers (memory-mapped peripherals)
- Variables modified by ISRs
- Variables shared between RTOS threads

```
volatile uint32_t tick_count = 0;

void TIM2_IRQHandler(void) {
    tick_count++; // Modified in ISR → must be volatile
}
```

Without `volatile`, the compiler at `-O2` may load `tick_count` into a CPU register once and never re-read from RAM — causing `main()` to never see updates from the ISR.

Common C Pitfalls (Embedded Edition)

Pitfall	What happens	Prevention
Uninitialized variable	Random RAM value used	Always initialize: <code>uint32_t x = 0;</code>
Integer overflow	<code>uint8_t 255 + 1 = 0</code>	Check bounds or use wider type

Pitfall	What happens	Prevention
NULL dereference	HardFault on Cortex-M	Check <code>ptr != NULL</code> before use
Buffer overrun	Stack/heap corruption	Always check <code>index < size</code>
Missing <code>volatile</code>	Compiler optimizes away reads	Use for ISR-shared and HW registers

STM32 Software Stack

What is Embedded Programming, Really?

GPIO, timers, UART, ADC, DMA — all just **reading and writing registers**. Registers live at fixed memory addresses defined by the chip. Every library (HAL, LL, CMSIS) eventually does the same thing: register access. The only question is: **how much help do you want** between your code and those registers?

Software Layers (Bottom-Up)

The STM32 software ecosystem has four layers between your code and the hardware:

1. **Hardware Registers** — Memory-mapped I/O at fixed addresses
2. **CMSIS** — ARM standard register structs (`GPIO_TypeDef`, `NVIC`). Maps struct fields to register offsets
3. **LL Drivers** — Lightweight wrapper functions over registers. No handles, minimal overhead
4. **HAL** — Portable, handle-based API with callbacks. CubeMX (separate tool) generates init code for this layer

Your application code sits on top and can call any layer directly.

CMSIS: Registers as Structs

CMSIS = Cortex Microcontroller Software Interface Standard (defined by ARM). It defines a C struct for each peripheral, mapping fields to register offsets:

```
typedef struct {
    __IO uint32_t MODER;    // offset 0x00 - Mode register
    __IO uint32_t OTYPER;   // offset 0x04 - Output type
    __IO uint32_t OSPEEDR;  // offset 0x08 - Output speed
    __IO uint32_t PUPDR;    // offset 0x0C - Pull-up/pull-down
    __IO uint32_t IDR;      // offset 0x10 - Input data (read-only)
    __IO uint32_t ODR;      // offset 0x14 - Output data
    // ... BSRR, LCKR, AFR[2]
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) 0x40020000)
```

GPIOA->ODR reads the 32-bit value at address $0x40020000 + 0x14 = 0x40020014$. This matches the datasheet register map exactly.

Direct Register Access via CMSIS

```
GPIOA->MODER |= (1 << 10);    // Set PA5 as output
GPIOA->ODR   |= (1 << 5);      // Set PA5 HIGH
GPIOA->ODR   &= ~(1 << 5);     // Set PA5 LOW

// Equivalent HAL call:
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
```

CMSIS gives typed access to every register. HAL gives a portable function that works across STM32 families.

HAL: Peripheral Handle Structures

Every HAL peripheral gets a handle struct containing configuration and state:

```
UART_HandleTypeDef huart2;
TIM_HandleTypeDef  htim2;
```

Inside a handle:

```
typedef struct {
    USART_TypeDef      *Instance; // → register block (e.g. USART2)
    UART_InitTypeDef    Init;      // baud rate, word length, parity...
    HAL_LockTypeDef     Lock;      // peripheral lock
    __IO HAL_UART_StateTypeDef State; // current state (ready/busy/error)
} UART_HandleTypeDef;
```

CubeMX generates these in `main.c`. You configure peripherals in CubeMX, it fills in the `Init` fields, HAL does the rest.

HAL Function Naming Convention

All HAL functions follow the pattern `HAL_<Peripheral>_<Action>()`:

Call	Meaning
HAL_GPIO_WritePin()	GPIO → Write a Pin
HAL_UART_Transmit()	UART → Transmit data
HAL_TIM_Base_Start_IT()	TIM → Start Base timer with Interrupt

Double-underscore macros compile to direct register writes:

- `__HAL_RCC_GPIOA_CLK_ENABLE()` — Enable GPIOA clock
- `__HAL_TIM_CLEAR_IT(&htim2, TIM_IT_UPDATE)` — Clear interrupt flag

Pattern: `Init` = one-time setup, `Start/Stop` = runtime control, `IRQHandler` = ISR entry.

HAL Callback Pattern

HAL uses **weak callbacks** — you override them with your own implementation:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        tick_count++;
    }
}
```

How the chain works:

1. Hardware interrupt fires → `TIM2_IRQHandler()` (in `stm32f4xx_it.c`)
2. HAL handler called → `HAL_TIM_IRQHandler(&htim2)`
3. HAL clears flags, identifies event, calls → `HAL_TIM_PeriodElapsedCallback()`
4. Your code runs inside the callback

Always check `htim->Instance` — all timers share the same callback function.

Common Callbacks

Peripheral	Callback	When it fires
Timer	<code>HAL_TIM_PeriodElapsedCallback()</code>	Timer overflow/update
UART	<code>HAL_UART_RxCpltCallback()</code>	Receive complete
UART	<code>HAL_UART_TxCpltCallback()</code>	Transmit complete
ADC	<code>HAL_ADC_ConvCpltCallback()</code>	Conversion done
SPI	<code>HAL_SPI_TxRxCpltCallback()</code>	Transfer complete

Choosing the Right Layer

Layer	Pros	Cons	Use when
HAL	Portable, readable, CubeMX generates init code	Overhead, large code size	Default choice for most code
LL	Lightweight, fast, vendor-supported	Less abstraction, no callbacks	Performance-sensitive paths
CMSIS	ARM standard, direct register access	Chip-specific, verbose	Understanding what HAL does
Bare register	Maximum control	Error-prone, not portable	Rarely — undocumented features only

This course: HAL (via CubeMX) for productivity, CMSIS registers to understand what's underneath. LL drivers appear in later weeks for time-critical code.

CubeMX → CubeIDE Workflow

1. Configure peripherals in **STM32CubeMX** (separate application)
2. Generate project code from CubeMX
3. Open the generated project in **STM32CubeIDE 2.0**
4. Write your application code on top of the generated code

```
/* main.c - generated by CubeMX, edited in CubeIDE */

/* USER CODE BEGIN Includes */
#include "my_app.h"          // Your includes here
/* USER CODE END Includes */

int main(void) {
    HAL_Init();
    SystemClock_Config();    // Generated - don't touch
    MX_GPIO_Init();          // Generated - don't touch
    MX_USART2_UART_Init();   // Generated - don't touch

    /* USER CODE BEGIN 2 */
    printf("Boot OK\r\n");   // Your init code here
```

```
/* USER CODE END 2 */

while (1) {
    /* USER CODE BEGIN 3 */
    // Your main loop here
    /* USER CODE END 3 */
}
}
```

Rule: Only write code between `USER CODE BEGIN` / `END` markers. Everything else gets overwritten when you regenerate from CubeMX.

Part 2: STM32F401 Peripheral Setup

STM32F401RE Overview

Feature	Value
Core	ARM Cortex-M4, 84 MHz
Flash / SRAM	512 KB / 96 KB
Timers	2x 32-bit, 4x 16-bit, 2x watchdog
ADC	1x 12-bit, 16 channels
Comm	3x USART, 3x SPI, 3x I2C, USB
GPIO	Up to 50 pins, 5V-tolerant on most
Board	Nucleo-F401RE (ST-LINK debugger built-in)

USART2 is connected to ST-LINK Virtual COM Port → USB → your terminal.

Clock Tree

The STM32F401 clock chain: HSI (16 MHz) → PLL → SYSCLK (84 MHz), then distributed to bus clocks:

- **AHB = 84 MHz** — GPIO, DMA
- **APB1 = 42 MHz** — TIM2-5, UART2, I2C, SPI2-3
- **APB2 = 84 MHz** — TIM1, TIM9-11, SPI1, ADC

Critical detail: APB1 timer clocks get a x2 multiplier, so TIM2 runs at 84 MHz (not 42 MHz).

You **must** enable the peripheral clock via RCC before accessing any register:

```
__HAL_RCC_GPIOA_CLK_ENABLE(); // Without this, GPIO reads return all zeros
```

GPIO Configuration

Four modes: Input, Output, Alternate Function (AF), Analog.

```
GPIO_InitTypeDef gpio = {0};
gpio.Pin      = GPIO_PIN_5;
gpio.Mode     = GPIO_MODE_OUTPUT_PP; // Push-pull output
gpio.Pull     = GPIO_NOPULL;
```

```
gpio.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &gpio);
```

GPIO Pin Modes

Mode	Use Case
Input (floating)	Default, susceptible to noise
Input (pull-up/down)	Buttons, switches
Output (push-pull)	LEDs, trace pins
Output (open-drain)	I2C, shared buses
Alternate function	UART, SPI, Timer
Analog	ADC input

Common Pitfalls

- Wrong AF number = silent failure (always check datasheet AF table)
- Forgetting `__HAL_RCC_GPIOx_CLK_ENABLE()` = dead pin
- Using a pin already assigned to another peripheral = conflict

Alternate Function Map (Key Pins)

Pin	AF7	AF1	AF2
PA2	USART2_TX	—	—
PA3	USART2_RX	—	—
PA5	—	TIM2_CH1	—
PA8	—	TIM1_CH1	—

CubeMX shows this visually in its pinout view. Always verify AF selection against the datasheet.

Timer Configuration

For 1 kHz interrupt on STM32F401RE:

$$f_{update} = \frac{f_{clock}}{(PSC + 1)(ARR + 1)} = \frac{84\,000\,000}{8400 \times 10} = 1000 \text{ Hz}$$

```
htim2.Instance = TIM2;
htim2.Init.Prescaler      = 8400 - 1;
htim2.Init.Period         = 10 - 1;
htim2.Init.CounterMode   = TIM_COUNTERMODE_UP;
HAL_TIM_Base_Init(&htim2);
HAL_TIM_Base_Start_IT(&htim2); // Enable update interrupt
```

Remember to enable TIM2 interrupt in NVIC (configure this in CubeMX before generating code).

UART Setup and printf Redirection

USART2 → PA2 (TX) / PA3 (RX) → ST-LINK VCP → USB → terminal

```
huart2.Instance      = USART2;
huart2.Init.BaudRate  = 115200;
huart2.Init.WordLength = UART_WORDLENGTH_8B;
huart2.Init.StopBits  = UART_STOPBITS_1;
huart2.Init.Parity     = UART_PARITY_NONE;
HAL_UART_Init(&huart2);
```

Redirect printf to UART:

```
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
```

Now `printf("Hello from STM32!\r\n");` appears in your serial terminal.

USART2 Quick Reference (Nucleo)

- TX: PA2 (AF7), RX: PA3 (AF7)
- Default: 115200 baud, 8N1
- Connected to ST-LINK virtual COM port

ISR Best Practices

Do:

- Toggle trace pin for timing measurement
- Set flags
- Increment counters
- Keep execution short ($< 10\ \mu\text{s}$ for 1 kHz rate)

Don't:

- Call printf or logging functions
- Allocate memory
- Run long loops
- Block on peripheral operations

```
void TIM2_IRQHandler(void) {  
    HAL_GPIO_WritePin(TRACE_GPIO_Port, TRACE_Pin, GPIO_PIN_SET);  
    tick_count++;  
    HAL_GPIO_WritePin(TRACE_GPIO_Port, TRACE_Pin, GPIO_PIN_RESET);  
    __HAL_TIM_CLEAR_IT(&htim2, TIM_IT_UPDATE);  
}
```

Part 3: Measurement Techniques

Trace Pin Method

1. Set GPIO HIGH at ISR entry
2. Set GPIO LOW at ISR exit
3. Measure with oscilloscope:
 - **Pulse width** = ISR execution time
 - **Period** = timer accuracy

Oscilloscope Settings

Parameter	Value	Why
Timebase	500 μ s/div	See ~2 periods of 1 ms signal
Voltage	1 V/div	3.3V logic fits in 4 divisions
Trigger	Rising edge, CH1	Stable capture

Measure: Period and pulse width (min/avg/max over 10+ cycles).

Troubleshooting Checklist

Nothing works?

1. Confirm clock enabled (`__HAL_RCC_<PERIPH>_CLK_ENABLE()`)
2. Verify pin AF in CubeMX configuration matches datasheet
3. Check NVIC enable for timer interrupt
4. Toggle GPIO in ISR to prove it runs

UART silent? Check baud rate, TX/RX not swapped, VCP driver installed.

Timer not firing? Check `HAL_TIM_Base_Start_IT()` was called, not just `Start()`.